

遊戲引擎 - AI 狩獵模擬與智慧角色決策系統

開發人員：唐予安

開發環境：C++17, OpenGL 4.5 (Core Profile), ImGui

核心架構：有限狀態機、視覺感測器、A* 尋路、導航網格、Context Steering、Last Known Position、回合制計分系統

1. 渲染效果與功能簡述

本專案實作一套 3D AI 狩獵模擬系統。場景設計為大型室內空間，內部包含多個房間、分隔牆、平台與障礙物。系統中共有兩種獵物與兩種捕食者，所有角色皆由 AI 自主控制，玩家只負責觀察與除錯，不會直接干涉角色行為。

本系統的核心目標是讓角色具備基本智慧與個性。獵物會在場景中漫遊，當發現捕食者時會逃跑；捕食者則會透過視覺感測尋找目標，並根據不同策略選擇追逐對象。當角色被牆壁遮擋時，捕食者會使用 A* 尋路繞行，或前往獵物最後被看見的位置進行搜查。

1.1 場景建置與狩獵環境

場景以大型室內房屋為基礎，外圍由牆面包圍，內部加入分隔牆與門洞，形成多房間結構。這樣的設計讓獵物與捕食者不能單純直線追逐，而必須受到視線遮擋、障礙物與尋路結果影響。

場景中也加入平台與障礙物，使 AI 需要處理繞行、卡住恢復與自動跳躍等問題。牆壁會同時影響碰撞、視線判定與導航網格，因此場景本身也是 AI 決策的重要條件。

1.2 獵物 AI 設計

本系統實作兩種獵物：

- 綠色普通獵物：價值 10 分，速度 3.0 m/s。
- 藍色敏捷獵物：價值 25 分，速度 4.5 m/s。

兩種獵物共用 `Wander / Flee` 狀態機，但藍色獵物速度較快，因此更難捕捉，也符合高價值獵物逃跑速度較快的作業要求。

獵物在沒有看見捕食者時會進入 Wander 狀態，在附近隨機選擇可行走位置進行漫遊。當獵物透過視覺感測發現捕食者後，會切換到 Flee 狀態。逃跑時並非單純往反方向移動，而是使用 Context Steering 評估多個方向，避開牆壁與障礙物。

簡化概念如下：

```
if (canSeePredator) {
    state = Flee;
    moveDirection = chooseBestFleeDirection();
}
else {
    state = Wander;
    moveDirection = moveToWanderTarget();
}
```

1.3 捕食者 AI 設計

本系統設計兩種捕食者，兩者速度相同，但目標選擇策略不同，因此具有不同個性。

捕食者 1：穩定短視型獵人

捕食者 1 採用最近鄰策略。當視野內有多個獵物時，會優先追逐距離最近的獵物。此策略穩定且容易捕捉附近目標，但不會考慮獵物分數。

```
target = findNearestVisiblePrey();
```

捕食者 2：貪婪高收益型獵人

捕食者 2 採用價值權重策略，會同時考慮獵物價值與距離。

```
Score = Value / (Distance + 0.1)
```

其中 `Value` 為獵物分數，`Distance` 為捕食者與獵物距離。此策略讓捕食者 2 不一定追最近的獵物，而是會在距離與收益之間做取舍。若藍色獵物距離不算太遠，捕食者 2 會傾向追逐高價值目標。

```
score = prey.value / (distance + 0.1f);
target = preyWithBestScore;
```

1.4 視覺感測器與感知系統

AI 的資訊來源以視覺感測為主，角色不能直接取得全場資訊。感知流程分成三個階段：

1. **距離檢測**：確認目標是否在視野半徑內。
2. **FOV 視野角檢測**：確認目標是否位於角色前方視野內。
3. **Line-of-Sight 遮擋檢測**：確認目標與 AI 之間是否被牆壁或障礙物阻擋。

簡化流程如下：

```
if (distance > visionRange) return false;
if (!insideFOV) return false;
if (lineOfSightBlocked) return false;

return true;
```

這個設計讓 AI 無法看穿牆壁。當獵物繞過牆角後，捕食者會失去目標視線，進而轉入最後已知位置調查或重新搜尋。

1.5 導航網格與 A* 尋路

為了讓 AI 能在多房間場景中移動，本專案實作 Navigation Grid 與 A* 尋路。系統在場景載入時會掃描靜態障礙物，並將牆壁與大型障礙物標記為不可通行區域。

A* 使用 4 方向搜尋，並以曼哈頓距離作為啟發式函數：

$$h(n) = \text{abs}(x1 - x2) + \text{abs}(z1 - z2)$$

當捕食者無法直接追逐目標，或目標被牆壁遮擋時，就會改用 A* 尋路前往目標附近或最後已知位置。這使捕食者能在多房間場景中繞過障礙物，而不是卡在牆後。

1.6 Last Known Position 搜查機制

若捕食者正在追逐獵物，但獵物突然被牆壁遮擋，捕食者不會立即放棄，而是會記錄獵物最後被看見的位置。接著捕食者會使用 A* 尋路前往該位置進行短暫搜查。

流程如下：

```
if (canSeeTarget) {
    lastKnownPosition = target.position;
```

```

        state = Chase;
    }
    else if (hasLastKnownPosition) {
        state = Investigate;
    }
    else {
        state = Search;
    }
}

```

此機制讓捕食者行為更自然，也避免 AI 在失去視線後立刻切回隨機搜尋。

1.7 角色運動與碰撞處理

AI 角色移動時會受到重力、牆壁碰撞與平台高度影響。系統不直接瞬移角色，而是根據速度更新位置，並在碰到牆壁時進行 push-out 修正與 sliding response。

當角色撞到牆壁時，系統會將角色推出牆體，並移除朝牆內的速度分量，使角色能沿牆面滑動。

```

position += normal * penetration;

if (dot(velocity, normal) < 0.0f) {
    velocity -= normal * dot(velocity, normal);
}

```

此外，若 AI 在牆角或門洞附近卡住，系統會透過 waypoint skipping 與 nudge 機制協助角色脫困，避免 AI 長時間停在同一位置。

1.8 捕捉判定與計分規則

本系統以距離作為捕捉判定。當捕食者與獵物之間距離小於 0.8f 時，即判定捕捉成功。

```

if (distance(predator, prey) < captureRadius) {
    prey.active = false;
    currentScore += prey.value;
}

```

捕捉成功後，獵物會被設為 inactive，並關閉顯示與碰撞，避免重複計分。綠色獵物提供 10 分，藍色獵物提供 25 分。

1.9 回合制模擬與勝負判定

本專案採用雙回合制比較兩種捕食者策略。第一回合釋放捕食者 1，第二回合釋放捕食者 2。兩回合會使用相同種子重置獵物位置，使兩個捕食者在相同條件下進行比較。

流程如下：

1. 生成 10 隻綠色獵物與 10 隻藍色獵物。
2. 第一回合由捕食者 1 進行限時狩獵。
3. 紀錄捕食者 1 分數。
4. 重置獵物位置。
5. 第二回合由捕食者 2 進行限時狩獵。
6. 紀錄捕食者 2 分數。
7. 比較兩者分數並宣告勝負。

```
if (score1 > score2) winner = "Predator 1 Wins";  
else if (score2 > score1) winner = "Predator 2 Wins";  
else winner = "Draw";
```

1.10 ImGui 與 Debug Visualization

ImGui 介面提供回合資訊、倒數計時、分數、勝負結果與參數調整功能。玩家可以重置模擬、切換回合、調整速度係數與視野半徑，也可以切換捕食者跟隨視角。

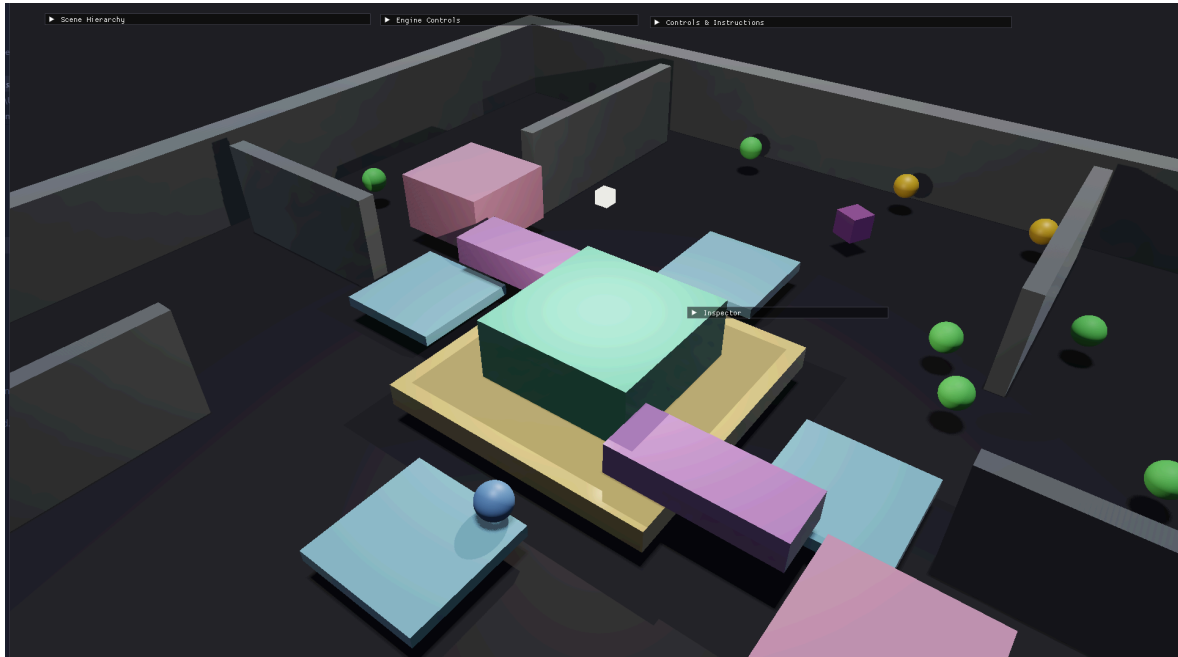
Debug Visualization 則用於顯示 AI 的內部狀態，例如：

- 導航網格阻擋區域。
- 視野錐體。
- A* 尋路路徑。
- 目標連線。
- Last Known Position。
- 當前追逐或逃跑目標。

這些功能主要用於觀察與除錯，不會直接干涉 AI 的決策結果。

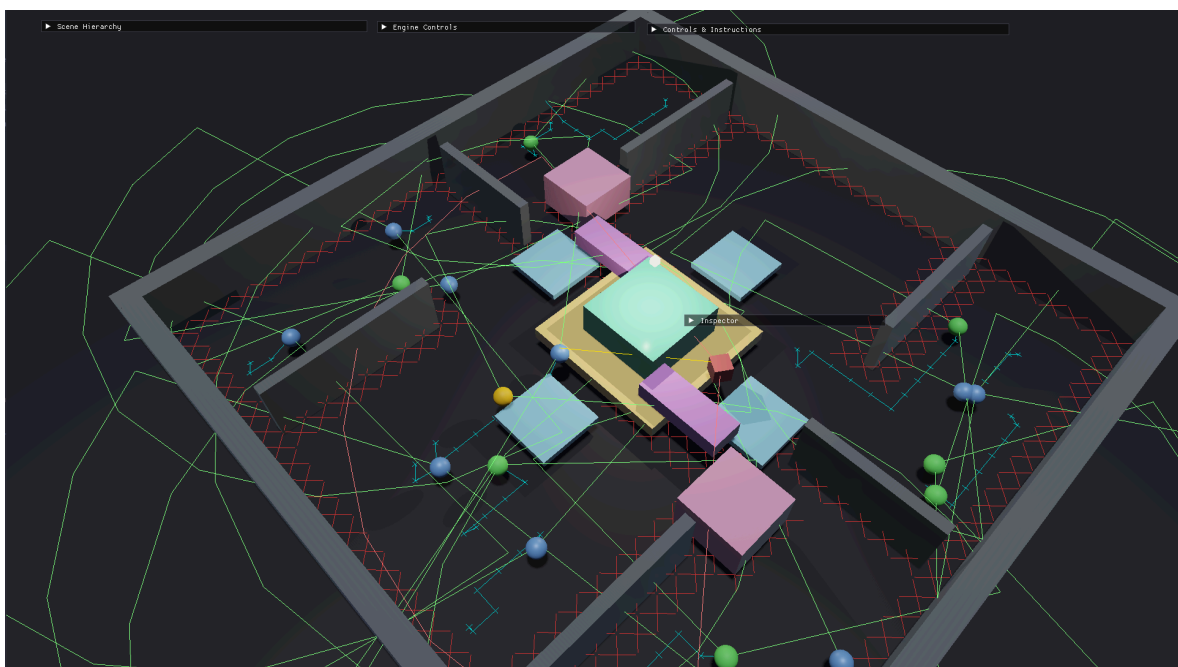
2. 程式執行內容與截圖說明

截圖內容說明（一）：大型室內狩獵場景



1. 畫面中可看到大型室內空間、外牆、內部分隔牆與平台。
2. 綠色與藍色獵物會分散在不同房間中自主移動。
3. 捕食者會依照目前回合被釋放進場。
4. 牆壁與障礙物會影響視線、碰撞與尋路結果。

截圖內容說明（二）：視覺感測與追逐



!AI_Perception_And_Chase.png

1. Debug 線條可顯示 AI 視野範圍。
 2. 當獵物進入捕食者視野後，捕食者會切換到 Chase 狀態。
 3. 若牆壁阻擋 Line-of-Sight，捕食者無法直接看見獵物。
 4. 有直接視線時，捕食者會直接朝目標追逐。
-

3. 操作方式說明

本程式主要為 AI 自動模擬，玩家不會直接控制獵物或捕食者。玩家操作僅用於觀察、除錯與參數調整。

3.1 視角控制

- **W / A / S / D**：控制相機移動。
- **Space**：相機向上移動。
- **Left Shift**：相機向下移動。
- **滑鼠移動**：旋轉觀察視角。
- **滑鼠滾輪**：調整觀察距離。
- **Follow Predator Camera**：切換捕食者跟隨視角。

3.2 ImGui 操作

- **Reset Simulation**：重置模擬。
 - **Next Round**：切換至下一回合。
 - **Prey Speed Scale**：調整獵物速度倍率。
 - **Predator Speed Scale**：調整捕食者速度倍率。
 - **Vision Range**：調整視覺感測距離。
 - **Debug Visualization**：顯示或隱藏 AI 除錯線條。
-

4. 模擬流程與資料結構設計

4.1 每個 Frame 的模擬流程

整體流程如下：

1. 更新回合計時器
2. 更新 AI 感知系統
3. 根據感知結果更新 FSM 狀態
4. 依照狀態決定移動目標
5. 必要時執行 A* 尋路
6. 更新角色速度、重力與碰撞
7. 檢查捕捉判定並更新分數
8. 若時間結束，切換回合或結算勝負
9. 渲染場景、AI、Debug 線條與 ImGui

4.2 Agent 資料結構

Agent 用來記錄獵物與捕食者的 AI 狀態。

```
struct Agent {  
    AgentType type;  
    AgentState state;  
  
    glm::vec3 position;  
    glm::vec3 velocity;  
    glm::vec3 forward;  
  
    float maxSpeed;  
    float visionRange;  
    float fovAngle;  
    int value;  
  
    bool active;  
  
    glm::vec3 targetPosition;  
    glm::vec3 lastKnownPosition;  
    std::vector<glm::vec3> path;  
};
```

主要用途包含記錄角色類型、目前狀態、位置、速度、視野參數、目標位置與尋路路徑。

4.3 NavigationGrid 資料結構

NavigationGrid 負責記錄可行走區域與不可通行區域，並提供 A* 尋路功能。


```

struct NavigationGrid {
    int width;
    int height;
    float cellSize;

    std::vector<GridCell> cells;

    bool isWalkable(int x, int z) const;
    glm::ivec2 worldToGrid(glm::vec3 pos) const;
    std::vector<glm::vec3> findPath(glm::vec3 start, glm::vec3 goal);
};

```

此結構能將世界座標轉換成格點座標，並根據場景障礙物建立尋路資料。

4.4 RoundSystem 資料結構

RoundSystem 用於管理回合、時間與分數。

```

struct RoundSystem {
    float timeRemaining;
    int predator1Score;
    int predator2Score;
    int currentRound;
};

```

此系統負責控制目前回合、倒數時間、分數累計與勝負判定。

5. 作業心得感想

這次 AI 引擎作業讓我理解到，遊戲 AI 並不是單純讓角色往目標移動，而是需要結合感知、決策、尋路、物理與遊戲規則。若只有追逐邏輯，角色很容易出現看穿牆壁、卡在角落或無法合理選擇目標的問題。因此這次我將視覺感測、FSM、A* 尋路與碰撞修正整合在一起，讓獵物與捕食者能在多房間場景中自主行動。

在捕食者設計上，我透過兩種不同策略表現不同個性。Predator 1 採用最近鄰策略，行為穩定且容易捕捉附近目標；Predator 2 則使用價值權重公式，在距離與分數之間做取舍。透過雙回合制與相同獵物分布，可以比較兩種 AI 策略在相同條件下的表現差異。

這次實作中最有收穫的部分是感知與尋路的整合。加入 Line-of-Sight 後，牆壁不再只是裝飾，而是真的會影響 AI 是否看見目標；加入 Last Known Position 後，捕食者失去視線時也不會立刻放棄，而是會前往最後看見的位置調查。整體而言，這次作業讓我更熟悉如何把多個 AI 子系統組合成完整的遊戲流程，也更理解 AI Debug Visualization 對分析角色行為的重要性。